CSC_52081_EP

Advanced Machine Learning and Autonomous Agents

Reinforcement Learning II

Jesse Read

Version: January 28, 2026

Outline

1. Quick Recap (MDP and Value)
2. Monte Carlo Methods for RL
   - MC Methods for Value Estimation
   - MC Methods for Control
   - Off-Policy MC
3. Temporal Difference (TD) Learning
   - TD
   - SARSA (On-Policy)
   - Q-Learning (Off-Policy)
4. Eligibility traces
5. Summary and Discussion

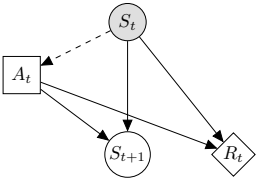
Roadmap for Today (28.01.2026)

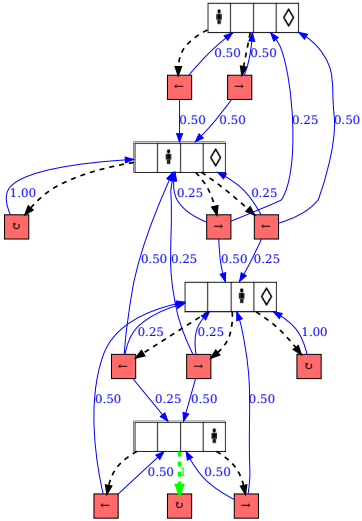
- Quick review of concepts: V, Q
- Monte Carlo Estimation for V
- Understand why we need Q
- Monte Carlo Estimation for Q
- Off-policy and importance sampling
- Issues with MC
- Temporal Difference (TD) Learning
- TD for V
- TD for Q – On policy and Off-policy
- Discussion: MC vs TD and MC / TD
- Limitations in the tabular setting.

Quick Recap (MDP and Value)

1. Quick Recap (MDP and Value)
2. Monte Carlo Methods for RL
   - MC Methods for Value Estimation
   - MC Methods for Control
   - Off-Policy MC
3. Temporal Difference (TD) Learning
   - TD
   - SARSA (On-Policy)
   - Q-Learning (Off-Policy)
4. Eligibility traces
5. Summary and Discussion

The Environment / Markov Decision Process (MDP)





- $\mathcal{S} = \{1, 2, 3, 4\}$
- $\mathcal{A} = \{\rightarrow, \circlearrowright, \leftarrow\}$
- $r(s, a) = \begin{cases} 1 & \text{when } (s, a) = (4, \circlearrowright) \\ 0 & \text{otherwise} \end{cases}$
- $p(s' \mid s, a)$
- $\pi \in \mathcal{A}(s)$ (actions available from s)

We may not have an expression for p and r!

The Gain (Return): (Discounted) Sum of Rewards from Current State

The gain (return)

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots$$
$$= \sum_{k=0}^{T} \gamma^k R_{t+k}$$

over horizon T (maybe, $T = \infty$) with discount factor $\gamma \in (0, 1)$.

The Policy

A deterministic policy

$$\pi : \mathcal{S} \to \mathcal{A}$$
$$\pi(s) \in \mathcal{A}(s)$$

e.g., the greedy policy

$$\pi(s) = \operatorname*{argmax}_{a \in \mathcal{A}} Q(s, a)$$

A stochastic policy

$$\pi : \mathcal{S} \times \mathcal{A} \to [0, 1]$$
$$\pi(a|s) \in [0, 1]$$

e.g., the ϵ-greedy policy

$$\pi(s) = \begin{cases} a^* & \text{with probability } 1 - \epsilon \\ a \sim \mathcal{A}(s) & \text{with probability } \epsilon \end{cases}$$

Value Functions

The state-value function

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

The action-value function

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

The value (expected gain) of policy π, performing in the environment, from state s [and taking initial action a from that state].

Tasks

Evaluation/Prediction Problem

$$V \approx v_\pi \quad \text{and/or} \quad Q \approx q_\pi$$

Evaluate/predict how policy π performs.

Control/Improvement Problem

$$\pi \rightsquigarrow \pi^*$$

Search for optimal policy π^*.

General schema of generalised policy improvement (GPI): policy-evaluation and policy-improvement processes interact; Bellman optimality equations: The interactive processes stabilizes/converges $\Leftrightarrow$ we have found optimal π^*, v^*.

Dynamic Programming

- Policy Evaluation ($V \approx v_\pi$)
- Value Iteration ($\pi \approx \pi_*$)
- Policy Iteration ($\pi \approx \pi_*$)

The Markov property allows for an efficient recursion; solve the MDP.

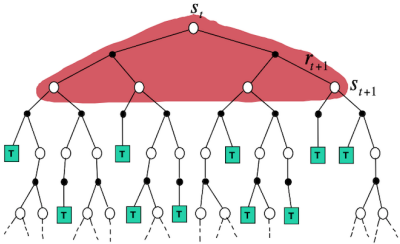


Image credits: David Silver's slides

However:

- Involves $|\mathcal{S}|$ equations $\Rightarrow$ expensive
- Requires knowledge of system dynamics $p(s', r \mid s, a)$
- Computing vs learning?

Monte Carlo Methods for RL

1. Quick Recap (MDP and Value)
2. Monte Carlo Methods for RL
   - MC Methods for Value Estimation
   - MC Methods for Control
   - Off-Policy MC
3. Temporal Difference (TD) Learning
   - TD
   - SARSA (On-Policy)
   - Q-Learning (Off-Policy)
4. Eligibility traces
5. Summary and Discussion

Monte Carlo Methods

Monte Carlo Methods are a broad class of algorithms based on repeatedly taking **samples**, solving any problem having a probabilistic interpretation, under the law of large numbers. We will use it to estimate

$$V(s) = \mathbb{E}[G \mid S = s]$$

and later, $Q(s, a) = \mathbb{E}[G \mid S = s, A = a]$.

In essence: We use MC methods to turn an $\mathbb{E}$ into a $\sum$.

Monte Carlo Methods for RL

1. Quick Recap (MDP and Value)
2. Monte Carlo Methods for RL
   - MC Methods for Value Estimation
   - MC Methods for Control
   - Off-Policy MC
3. Temporal Difference (TD) Learning
   - TD
   - SARSA (On-Policy)
   - Q-Learning (Off-Policy)
4. Eligibility traces
5. Summary and Discussion

Monte Carlo for Evaluation/Prediction ($V \approx v_\pi$)

Recall the value function for policy π:

$$v^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

Monte Carlo: Replace the expectation with average over samples.

Samples come from a trajectory (rollout, episode); π interacting with environment p:

$$\underbrace{S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_T, A_T, R_T}_{\tau} \sim p_\pi$$

Basic recipe for evaluation (from π to v_π):

- Play many episodes with π
- Record actual returns from visit[1] to each s.
- Return average as approximation of $v_\pi(s)$

[1] Can be first visit or every visit

First-Visit Monte Carlo for Prediction

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

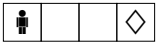
Unless S_t appears in $S_0, S_1, \ldots, S_{t-1}$:

Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

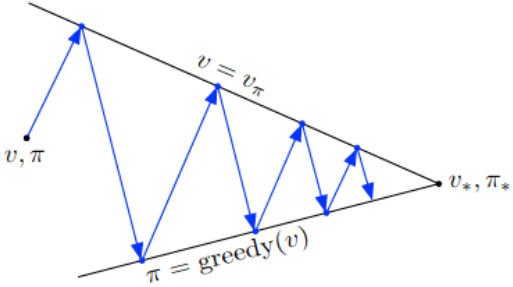
From Sutton & Barto, 2020

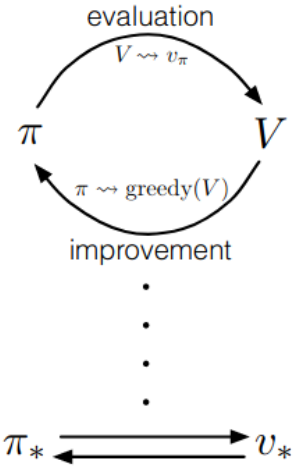
Example: Toy World



$s =$	1	2	3	4
$v_\pi =$				

Generalised Policy Iteration

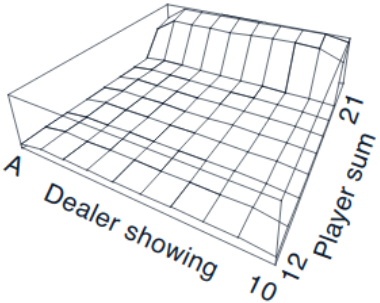




Ref: Sutton and Barto. *Reinforcement Learning: An Introduction*, 2020.

Example: Blackjack

v^π for policy π that sticks on 20 or 21:

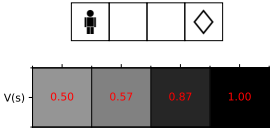


From Sutton & Barto, 2020: Blackjack, $V(s)$ with no usable ace; after 500,000 episodes

The agent has learned the game dynamics *indirectly* via V

But: How to play now (what is π)?

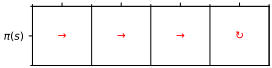
V-Learning or Q-Learning?



Only useful if our agent knows how the environment works! Note:

	←	↻	→
i = 1	0.06	0.12	0.50
i = 2	0.33	0.34	0.57
i = 3	0.31	0.39	0.87
i = 4	0.56	1.00	0.60

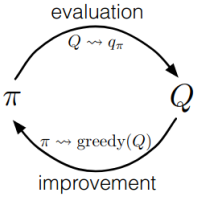
The Q-function tells us the action to take!

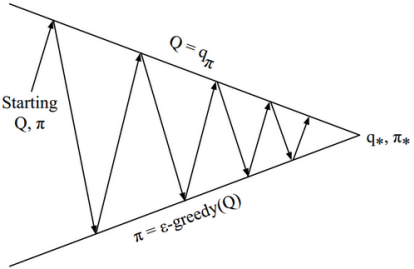


Monte Carlo Methods for RL

1. Quick Recap (MDP and Value)
2. Monte Carlo Methods for RL
   - MC Methods for Value Estimation
   - MC Methods for Control
   - Off-Policy MC
3. Temporal Difference (TD) Learning
   - TD
   - SARSA (On-Policy)
   - Q-Learning (Off-Policy)
4. Eligibility traces
5. Summary and Discussion

Monte Carlo Control ($\pi \rightsquigarrow \pi^*$)





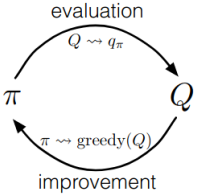
From Sutton & Barto, 2020

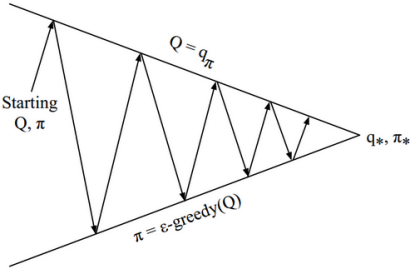
Recall the greedy policy for a given q:

$$\pi(s) = \underset{a \in \mathcal{A}(s)}{\operatorname{argmax}}\, q(s, a)$$

$$\pi_0 \to q_{\pi_0} \to \pi_1 \to q_{\pi_1} \to \cdots \to \pi_k \to q_{\pi_k} \to \cdots \to \pi_*$$

Monte Carlo Control ($\pi \rightsquigarrow \pi^*$)





From Sutton & Barto, 2020

Recall the greedy policy for a given q:

$$\pi(s) = \underset{a \in \mathcal{A}(s)}{\operatorname{argmax}}\, q(s, a)$$

$$\pi_0 \to q_{\pi_0} \to \pi_1 \to q_{\pi_1} \to \cdots \to \pi_k \to q_{\pi_k} \to \cdots \to \pi_*$$

Problem: With the greedy policy we typically lack observations for many (s, a)-pairs.

Encouraging Exploration via ϵ-soft Policies and ϵ-greedy

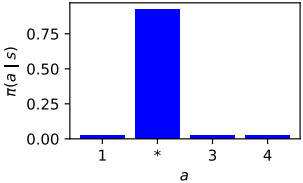
An ϵ-soft policy is *any* policy where

$$\pi(a \mid s) > 0 \quad \text{for all } s \in \mathcal{S} \text{ and } a \in \mathcal{A}(s)$$

For example: The ϵ-greedy policy; select optimal action a^* (according to current knowledge) with probability $1 - \epsilon$, and random action with probability ϵ.

$$\pi(a^{\neg *} \mid s) = \frac{\epsilon}{|\mathcal{A}(s)|} \quad \text{and} \quad \pi(a^* \mid s) = (1 - \epsilon) + \frac{\epsilon}{|\mathcal{A}(s)|}$$

For example (where $\epsilon = 0.1$):



Other options to encourage exploration: use a softmax policy, do exploring starts, or initialize the value functions with high values.

On-policy first-visit MC control (for ε-soft policies), estimates $\pi \approx \pi_*$

Algorithm parameter: small $\varepsilon > 0$

Initialize:

$\pi \leftarrow$ an arbitrary ε-soft policy

$Q(s,a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

$Returns(s,a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Repeat forever (for each episode):

Generate an episode following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

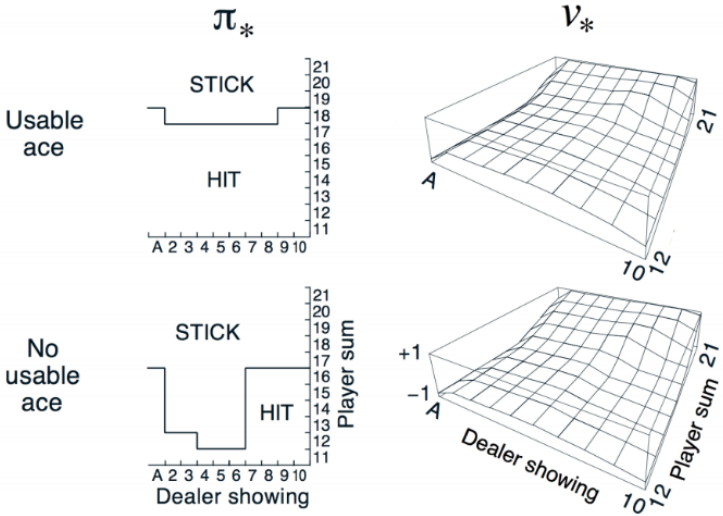
$A^* \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken arbitrarily)

For all $a \in \mathcal{A}(S_t)$:

$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \varepsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

From Sutton & Barto, 2020

Example: Playing Blackjack



From *Sutton & Barto, 2020*. The state-value function v_* (right) has been calculated from action-value function q_*, which determines π_* (left).

Monte Carlo Methods for RL

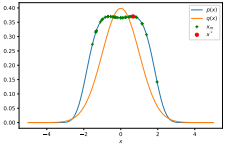
1. Quick Recap (MDP and Value)
2. Monte Carlo Methods for RL
   - MC Methods for Value Estimation
   - MC Methods for Control
   - Off-Policy MC
3. Temporal Difference (TD) Learning
   - TD
   - SARSA (On-Policy)
   - Q-Learning (Off-Policy)
4. Eligibility traces
5. Summary and Discussion

Off-policy Monte-Carlo with Importance Sampling

Practical issue: we seek optimal behaviour, but are forced to behave non-optimally in order to do so (i.e., to explore)!

Consider off-policy (two-policy) methods: one policy for exploration (behavioural policy), another for exploitation (target policy).

Requires additional concepts; importance sampling.



Example: Off-Policy Monte Carlo Prediction

Again, we want expected gain,

$$v^{\pi}(s) = \mathbb{E}_{\mu}[G_t \mid S_t = s]$$

but this time, via behavioural policy μ.

Record trajectory $\tau_m = \{s_t = s, a_t, r_t, s_{t+1}, a_{t+1}, \ldots, r_T\} \sim \mu$ from state s at time t.
Approximate the expectation:

$$V(s) = \frac{1}{M} \sum_{m=1}^{M} \omega_m G_t^{(m)} \approx \mathbb{E}[G_t | S_t = s]$$

with importance weight (to compensate the fact that $\pi \neq \mu$):

$$\begin{aligned}
\omega_m &= \frac{p_{\pi}(\tau_m)}{p_{\mu}(\tau_m)} \\
&= \frac{\prod_{k=t}^{T-1} \pi(a_k|s_k) \cancel{p(s_{k+1}|s_k, a_k)}}{\prod_{k=t}^{T-1} \mu(a_k|s_k) \cancel{p(s_{k+1}|s_k, a_k)}} = \prod_{k=t}^{T-1} \frac{\pi(a_k|s_k)}{\mu(a_k|s_k)}
\end{aligned}$$

Off-policy MC control, for estimating $\pi \approx \pi_*$

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:
- $Q(s,a) \in \mathbb{R}$ (arbitrarily)
- $C(s,a) \leftarrow 0$
- $\pi(s) \leftarrow \operatorname{argmax}_a Q(s,a)$ (with ties broken consistently)

Loop forever (for each episode):
- $b \leftarrow$ any soft policy
- Generate an episode using b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
- $G \leftarrow 0$
- $W \leftarrow 1$
- Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
  - $G \leftarrow \gamma G + R_{t+1}$
  - $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
  - $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)]$
  - $\pi(S_t) \leftarrow \operatorname{argmax}_a Q(S_t, a)$ (with ties broken consistently)
  - If $A_t \neq \pi(S_t)$ then exit inner Loop (proceed to next episode)
  - $W \leftarrow W \frac{1}{b(A_t|S_t)}$

From Sutton & Barto, 2020

Monte Carlo Reinforcement Learning: Summary

- No prior knowledge about the environment dynamics is necessary
- Use samples (empirical episodes) from the environment
- We obtain unbiased estimates $V(s)$ (or $Q(s, a)$)
- After infinite samples, we converge!

but,

- Observing empirical return implies that the agent has to wait until the end of an episode to improve the policy
- High variance
- Infinite samples is a lot

i.e., can take a long time to learn.

Temporal Difference (TD) Learning

1. Quick Recap (MDP and Value)
2. Monte Carlo Methods for RL
   - MC Methods for Value Estimation
   - MC Methods for Control
   - Off-Policy MC
3. Temporal Difference (TD) Learning
   - TD
   - SARSA (On-Policy)
   - Q-Learning (Off-Policy)
4. Eligibility traces
5. Summary and Discussion

Wikipedia.

Temporal Difference (TD) Learning

1. Quick Recap (MDP and Value)
2. Monte Carlo Methods for RL
   - MC Methods for Value Estimation
   - MC Methods for Control
   - Off-Policy MC
3. Temporal Difference (TD) Learning
   - TD
   - SARSA (On-Policy)
   - Q-Learning (Off-Policy)
4. Eligibility traces
5. Summary and Discussion

The Temporal Difference

Recall: V is an estimate of v_π or v_*.

$$\begin{aligned} V(S_t) &= R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots \\ &= R_t + \gamma V(S_{t+1}) \end{aligned}$$

The Temporal Difference

Recall: V is an estimate of v_π or v_*.

$$\begin{aligned} V(S_t) &= R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots \\ &= R_t + \gamma V(S_{t+1}) \\ 0 &= R_t + \gamma V(S_{t+1}) - V(S_t) \quad ??? \end{aligned}$$

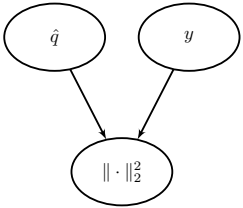
In practice, is not the case!

$$\underbrace{\underbrace{[R_t + \gamma V(S_{t+1})]}_{\text{target (future) } V} - \underbrace{V(S_t)}_{\text{current } V}}_{\text{The temporal difference; } \delta} \neq 0$$

TD Learning: Learn from the future; reduce the TD error (δ) toward 0

Learning (Minimising the TD error)

$$
\begin{aligned}
y &= R_t + \gamma V(S_{t+1}) \quad \triangleright \text{ Target} \\
\hat{q} &= V(S_t) \quad \triangleright \text{ Current estimate} \\
\mathcal{L} &= \frac{1}{2}(y - \hat{q})^2 \quad \triangleright \text{ Loss} \\
\nabla_{\hat{q}} \mathcal{L} &= \nabla_{\hat{q}} \frac{1}{2}(y - \hat{q})^2 \\
&= (y - \hat{q}) \nabla_{\hat{q}} (y - \hat{q}) \\
&= \hat{q} - y
\end{aligned}
$$



Computational graph

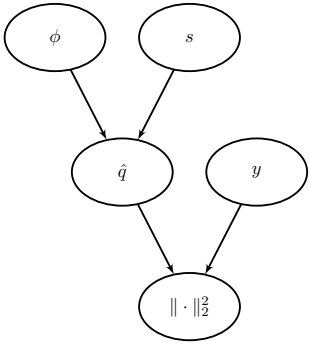
We can do updates (gradient descent):

$$V(S_t) \leftarrow V(S_t) + \alpha \left([R_t + \gamma V(S_{t+1})] - V(S_t) \right)$$

with learning rate α; upon each new S_{t+1}.

Learning (Minimising the TD error) – Deep Learning Version

$$
\begin{aligned}
y &= R_t + \gamma q_\phi(s_{t+1}) \quad \triangleright \text{ Target} \\
q_\phi(s_t) &= \quad \triangleright \text{ Current estimate} \\
\nabla_\phi \mathcal{L} &= \nabla_\phi \frac{1}{2}(y - q_\phi)^2 \quad \triangleright \text{ Loss} \\
&= (y - q_\phi)\nabla_\phi(y - q_\phi) \\
&= (q_\phi - y)\nabla_\phi q_\phi
\end{aligned}
$$



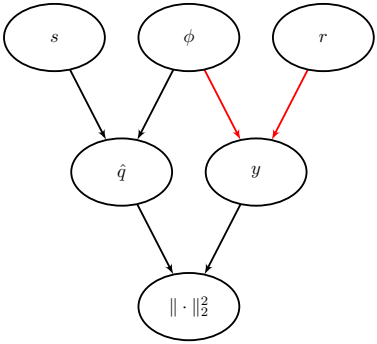
Computational graph

Training data: $\mathcal{D} = \{(\hat{q}^{(i)}, y^{(i)})\}_{i=1}^N$, i.e., target $y^{(i)}$

Take a moment to think about how unusual this is (from ML perspective)!
(We'll discuss more later).

Learning (Minimising the TD error) – Deep Learning Version

$$
\begin{aligned}
y &= R_t + \gamma q_\phi(s_{t+1}) \quad \triangleright \text{ Target} \\
q_\phi(s_t) &= \quad \triangleright \text{ Current estimate} \\
\nabla_\phi \mathcal{L} &= \nabla_\phi \frac{1}{2}(y - q_\phi)^2 \quad \triangleright \text{ Loss} \\
&= (y - q_\phi)\nabla_\phi(y - q_\phi) \\
&= (q_\phi - y)\nabla_\phi q_\phi
\end{aligned}
$$



Computational graph

Training data: $\mathcal{D} = \{(\hat{q}^{(i)}, y^{(i)})\}_{i=1}^N$, i.e., target $y^{(i)}$

Take a moment to think about how unusual this is (from ML perspective)!
(We'll discuss more later).

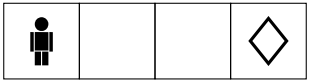
TD for Prediction (Estimating $V \approx v_\pi$)

Tabular TD(0) for estimating v_π

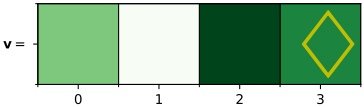
Input: the policy π to be evaluated
Algorithm parameter: step size $\alpha \in (0, 1]$
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    $A \leftarrow$ action given by π for S
    Take action A, observe R, S'
    $V(S) \leftarrow V(S) + \alpha\big[R + \gamma V(S') - V(S)\big]$
    $S \leftarrow S'$
  until S is terminal

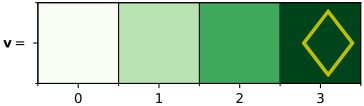
From Sutton & Barto, 2020. TD(0) $\Rightarrow$ one-step ahead. $\mathcal{S}^+$ means all states including the terminal state.



Environment



V via TD at $t = 0$



V via TD at $t = 100$

TD learning uses an estimate to compute next estimate;

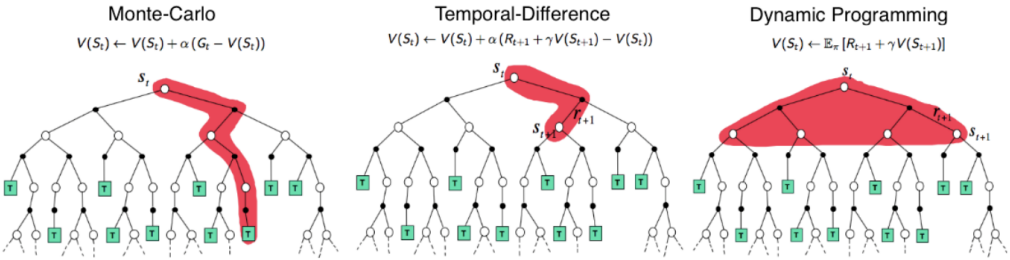
- a form of bootstrapping,
- introduces bias,
- related to dynamic programming (and Monte Carlo)
- Related to the way dopamine cells operate in animal learning[2]

Wikipedia.

[2] Hollerman and Schultz. *Dopamine neurons report an error in the temporal prediction of reward during learning*, 1998.
First mention in CS: Witten, I. H. *An adaptive optimal for discrete-time Markov environments*, 1977.

TD vs Monte Carlo (MC), Dynamic Programming (DP)

- DP leverages recursion on known dynamics p to calculate $\mathbb{E}_\pi[G_t \mid S_t = s]$
- MC estimates $\mathbb{E}_\pi[G_t \mid S_t = s]$ with full-episode samples
  - No knowledge of model required (unlike DP methods)
- TD samples just one-step and uses current estimate V
  - No need to wait to end of episode; can be implemented online (unlike MC methods)



From Sutton & Barto, 2020

MC replaces $\mathbb{E}$ with $\sum$; and TD replaces $\sum$ with single step.

TD for Control: Learning $Q(S_t, A_t) \approx q_\pi(s, a)$

TD estimation:

$$V(S_t) \leftarrow V(S_t) + \alpha \left[(R_t + \gamma V(S_{t+1})) - V(S) \right]$$

To know which action to take, we need either knowledge of the MDP or... to learn Q.

Off-policy TD control (**Q-Learning**):

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha \left[(R_t + \gamma \max_{a \in \mathcal{A}} Q(S_{t+1}, a)) - Q(S_t, A_t) \right]$$

On-Policy TD control (**SARSA**):

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha \left[(R_t + \gamma Q(S_{t+1}, A_{t+1})) - Q(S_t, A_t) \right]$$

Note: SARSA is also Q-Learning. 'Q-Learning' could be called 'SARS' or 'SARSa'.

Temporal Difference (TD) Learning

1. Quick Recap (MDP and Value)
2. Monte Carlo Methods for RL
   - MC Methods for Value Estimation
   - MC Methods for Control
   - Off-Policy MC
3. Temporal Difference (TD) Learning
   - TD
   - SARSA (On-Policy)
   - Q-Learning (Off-Policy)
4. Eligibility traces
5. Summary and Discussion

SARSA (On-Policy TD Control)

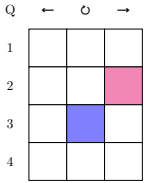
SARSA is an on-policy TD method for learning Q. Its name comes from the trajectories

$$\ldots, S_t\, A_t\, R_t\, S_{t+1}\, A_{t+1}, \ldots$$

(generated by policy π interacting with environment p).

The SARSA TD update:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_t + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\right]$$



Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\mathit{terminal}, \cdot) = 0$

Loop for each episode:
  Initialize S
  Choose A from S using policy derived from Q (e.g., ε-greedy)
  Loop for each step of episode:
    Take action A, observe R, S'
    Choose A' from S' using policy derived from Q (e.g., ε-greedy)
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$
    $S \leftarrow S'$; $A \leftarrow A'$;
  until S is terminal

From Sutton & Barto, 2020

Temporal Difference (TD) Learning

1. Quick Recap (MDP and Value)
2. Monte Carlo Methods for RL
   - MC Methods for Value Estimation
   - MC Methods for Control
   - Off-Policy MC
3. Temporal Difference (TD) Learning
   - TD
   - SARSA (On-Policy)
   - Q-Learning (Off-Policy)
4. Eligibility traces
5. Summary and Discussion

Q-Learning (Off-policy TD Control)

Q-Learning is a TD method, or class of methods (including SARSA) for learning Q. We refer here to vanilla off-policy Q-Learning.

Small difference from SARSA, ...

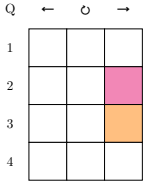
$$\ldots, S_t\ A_t\ R_t\ S_{t+1}\ a, \ldots$$

where the update is:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_t + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

It means that

$$A_t \sim \pi_{\epsilon\text{-greedy}}(S_t) \qquad \text{and} \qquad a = \pi_{\text{greedy}}(S_{t+1})$$



Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Take action A, observe R, S'
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma \max_a Q(S', a) - Q(S, A)\big]$
    $S \leftarrow S'$
  until S is terminal

From Sutton & Barto, 2020

Off-Policy vs On-Policy TD Control

Why is Q-learning an off-policy method? (Why is SARSA on-policy?)

Both policies *follow* (generate data

$$\ldots, S_t\, A_t\, R_t\, S_{t+1}\, A_{t+1}, \ldots$$

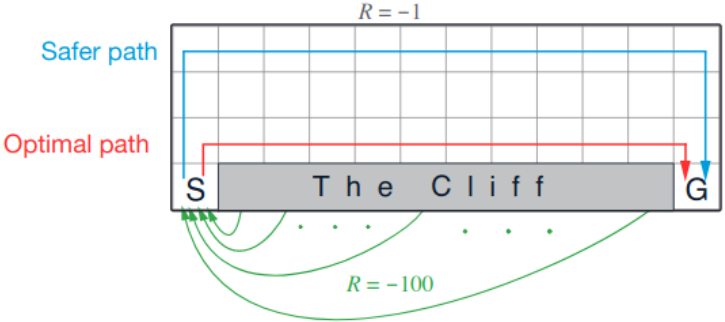
using) policy π_t (ϵ-soft). But Q-learning uses a different policy (greedy) to determine value:

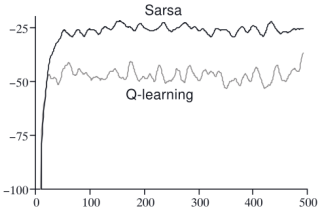
$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big[R_t + \gamma \underbrace{\max_a Q(S_{t+1}, a)}_{\text{via } \tilde{\pi}_t^* \text{(greedy)}} - Q(S_t, A_t)\big]$$

So in Q-Learning we learn $\tilde{\pi}_t^* \rightsquigarrow \pi^*$, rather than $\pi_t \rightsquigarrow \pi^*$.

Off-policy without importance sampling? No probability/policy-distribution (just a max) – no correction needed.

SARSA vs Q-Learning: The Cliff Example





From Sutton & Barto, 2020

Another Problem with Q-Learning: Maximisation Bias

Suppose $Q^*(s, a) = 0$ for s and all a. But suppose:

$$\hat{Q}(s, a_j) \sim \mathcal{N}(0, \sigma^2)$$

We can show that

$$\mathbb{E}[\max(Q(s, a_1), \ldots, Q(s, a_{10}))] > 0$$

which is incorrect. The max causes us to systematically over-estimate the value (even when all actions are worthless).

Another Problem with Q-Learning: Maximisation Bias

Suppose $Q^*(s, a) = 0$ for s and all a. But suppose:

$$\hat{Q}(s, a_j) \sim \mathcal{N}(0, \sigma^2)$$

We can show that

$$\mathbb{E}[\max(Q(s, a_1), \ldots, Q(s, a_{10}))] > 0$$

which is incorrect. The max causes us to systematically over-estimate the value (even when all actions are worthless).

Bias, highly suboptimal performance during learning, ... why choose Q-learning (over SARSA or MC methods)?

Another Problem with Q-Learning: Maximisation Bias

Suppose $Q^*(s, a) = 0$ for s and all a. But suppose:

$$\hat{Q}(s, a_j) \sim \mathcal{N}(0, \sigma^2)$$

We can show that

$$\mathbb{E}[\max(Q(s, a_1), \ldots, Q(s, a_{10}))] > 0$$

which is incorrect. The max causes us to systematically over-estimate the value (even when all actions are worthless).

Bias, highly suboptimal performance during learning, ... why choose Q-learning (over SARSA or MC methods)?

1. The bias is by design! Q-Learning is optimistic by construction.
   - Recall: bias-variance tradeoff (MC, by contrast: higher variance).
   - We care about *optimising* value, not estimating value.
2. Learning may be 'riskier' (than, e.g., SARSA) but it is usually faster!
3. As an off-policy method, we can use a reply buffer (we will come back to this later) without 'breaking' anything, and without need for [explicit] importance sampling.

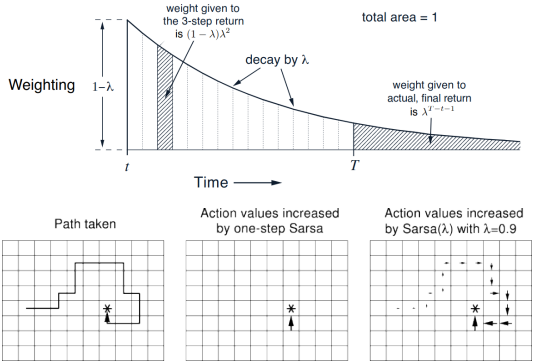
Eligibility traces

1. Quick Recap (MDP and Value)
2. Monte Carlo Methods for RL
   - MC Methods for Value Estimation
   - MC Methods for Control
   - Off-Policy MC
3. Temporal Difference (TD) Learning
   - TD
   - SARSA (On-Policy)
   - Q-Learning (Off-Policy)
4. Eligibility traces
5. Summary and Discussion

Eligibility Traces: TD(λ)

Problem of TD(0): information takes a long time to reach all the states (or might not even reach them) in a large state space.

Solution: weighting factor λ.



From Sutton & Barto, 2020

A unified view: TD(λ) is between 1-step TD(0) and MC (TD(1)).

Summary and Discussion

1. Quick Recap (MDP and Value)
2. Monte Carlo Methods for RL
   - MC Methods for Value Estimation
   - MC Methods for Control
   - Off-Policy MC
3. Temporal Difference (TD) Learning
   - TD
   - SARSA (On-Policy)
   - Q-Learning (Off-Policy)
4. Eligibility traces
5. Summary and Discussion

Summary: Value-based Learning

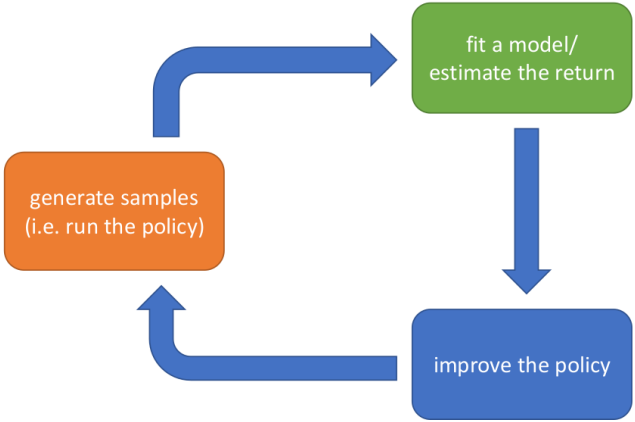


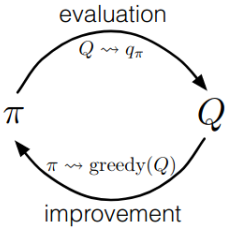
Image: Sergey Levine

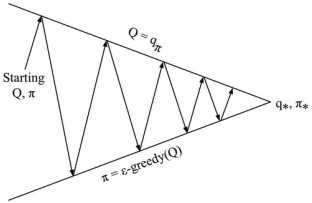
Fit $Q(s, a)$, then set $\pi(s) = \text{argmax}_a Q(s, a)$, then generate $\tau \sim \pi$.

Generalised Policy Improvement: The policy is at least as good as the previous one!

Summary: MC and TD

Monte Carlo (MC) and Temporal Difference (TD) learning.





Estimation: evaluate the value function; $V(s)$

Control: optimize the value function; $Q(s, a)$

- on-policy (learn π and follow π): SARSA
- off-policy (learn π^*, follow π_t): Q-Learning

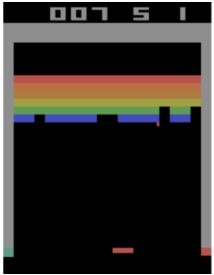
Combining both: TD(λ) where $\lambda = 1 \Leftrightarrow MC$!

MC vs TD

	TD	MC
Bias	Biased	Unbiased
Variance	Low	High
Deterministic	No problem	Degenerate
Updates	Stepwise	Episode-wise
Implementation	Simple	Simple
Markov	Strictly required	Less harmed by deviations
Main plus	Better on long episodes	Clean statistical reasoning

Limitations and Upcoming Material

Producing and updating a Q-table (and associated exploration) is tricky with a big state space/many possible state-action pairs.



We started to see that using deep neural architectures made things a bit weird, from a Deep ML perspective. Over the next weeks, we will look at:

- Deep Q-Learning and variants (continuing 'value-based RL')
- Deep Policy Gradient methods (looking at 'policy-based RL')
- Deep Actor-Critic methods and variants (combining ideas from both)

CSC_52081_EP

Advanced Machine Learning and Autonomous Agents

Reinforcement Learning II

Jesse Read

Version: